



Exercise 5: Simulate the Page Frame Replacement

Lappeenranta–Lahti University of Technology LUT

Computational Science and Artificial Intelligence

CT30A3350 Operating Systems and System Programming (Lahti)

Date: 15.04.2026

Md Hamidur Rahman Khan, 002475552

Examiner: Ankit Agrawal, PhD

The number of virtual pages was 7, the number of physical page frames was 4 and the page reference string was: 1, 2, 3, 4, 2, 6, 2, 1, 2, 3, 7, 6, 3, 2, 1, 2, 3, 6.

FIFO page replacement result

```
hamidur@DESKTOP-7CGRQ8S:~/LUT_exercises/os_sp/ex5$ gcc -o page_replacement page_replacement.c
hamidur@DESKTOP-7CGRQ8S:~/LUT_exercises/os_sp/ex5$ ./page_replacement
Please choose page replacement algorithm: 1.FIFO 2.LRU 3.OPT 0.quit
1

FIFO page replacement
Access page 1: Page fault. Frames: [1] [ ] [ ] [ ]
Access page 2: Page fault. Frames: [1] [2] [ ] [ ]
Access page 3: Page fault. Frames: [1] [2] [3] [ ]
Access page 4: Page fault. Frames: [1] [2] [3] [4]
Access page 2: Hit. Frames: [1] [2] [3] [4]
Access page 6: Page fault. Frames: [6] [2] [3] [4]
Access page 2: Hit. Frames: [6] [2] [3] [4]
Access page 1: Page fault. Frames: [6] [1] [3] [4]
Access page 2: Page fault. Frames: [6] [1] [2] [4]
Access page 3: Page fault. Frames: [6] [1] [2] [3]
Access page 7: Page fault. Frames: [7] [1] [2] [3]
Access page 6: Page fault. Frames: [7] [6] [2] [3]
Access page 3: Hit. Frames: [7] [6] [2] [3]
Access page 2: Hit. Frames: [7] [6] [2] [3]
Access page 1: Page fault. Frames: [7] [6] [1] [3]
Access page 2: Page fault. Frames: [7] [6] [1] [2]
Access page 3: Page fault. Frames: [3] [6] [1] [2]
Access page 6: Hit. Frames: [3] [6] [1] [2]
The number of page faults of FIFO is: 13
Page fault rate: 0.7222
The number of replacements: 9
Replacement rate: 0.5000
0
hamidur@DESKTOP-7CGRQ8S:~/LUT_exercises/os_sp/ex5$
```

Figure 1. Terminal showing compile and run command with the resulting output of FIFO page replacement

FIFO removes the page that has been in memory the longest, without considering if it will be needed again soon. Because of that, a page that is still useful can be replaced too early.

In this case, the result was 13 page faults and a page fault rate of 0.7222. The algorithm also made 9 replacements and the replacement rate was 0.5000.

LRU page replacement result

```
hamidur@DESKTOP-7CGRQ8S:~/LUT_exercises/os_sp/ex5$ ./page_replacement
Please choose page replacement algorithm: 1.FIFO 2.LRU 3.OPT 0.quit
2

LRU page replacement
Access page 1: Page fault. Frames: [1] [ ] [ ] [ ]
Access page 2: Page fault. Frames: [1] [2] [ ] [ ]
Access page 3: Page fault. Frames: [1] [2] [3] [ ]
Access page 4: Page fault. Frames: [1] [2] [3] [4]
Access page 2: Hit. Frames: [1] [2] [3] [4]
Access page 6: Page fault. Frames: [6] [2] [3] [4]
Access page 2: Hit. Frames: [6] [2] [3] [4]
Access page 1: Page fault. Frames: [6] [2] [1] [4]
Access page 2: Hit. Frames: [6] [2] [1] [4]
Access page 3: Page fault. Frames: [6] [2] [1] [3]
Access page 7: Page fault. Frames: [7] [2] [1] [3]
Access page 6: Page fault. Frames: [7] [2] [6] [3]
Access page 3: Hit. Frames: [7] [2] [6] [3]
Access page 2: Hit. Frames: [7] [2] [6] [3]
Access page 1: Page fault. Frames: [1] [2] [6] [3]
Access page 2: Hit. Frames: [1] [2] [6] [3]
Access page 3: Hit. Frames: [1] [2] [6] [3]
Access page 6: Hit. Frames: [1] [2] [6] [3]
The number of page faults of LRU is: 10
Page fault rate: 0.5556
The number of replacements: 6
Replacement rate: 0.3333
0
hamidur@DESKTOP-7CGRQ8S:~/LUT_exercises/os_sp/ex5$
```

Figure 2. Terminal showing run command with the resulting output of LRU page replacement

LRU performed better than FIFO because it considers recent page usage. It replaces the page that has not been used for the longest time, which usually matches the actual access pattern better than FIFO.

In this case, LRU resulted in 10 page faults, so the page fault rate was 0.5556. It also caused 6 replacements and a replacement rate of 0.3333.

OPT page replacement result

```
hamidur@DESKTOP-7CGRQ8S:~/LUT_exercises/os_sp/ex5$ ./page_replacement
Please choose page replacement algorithm: 1.FIFO 2.LRU 3.OPT 0.quit
3

OPT page replacement
Access page 1: Page fault. Frames: [1] [ ] [ ] [ ]
Access page 2: Page fault. Frames: [1] [2] [ ] [ ]
Access page 3: Page fault. Frames: [1] [2] [3] [ ]
Access page 4: Page fault. Frames: [1] [2] [3] [4]
Access page 2: Hit. Frames: [1] [2] [3] [4]
Access page 6: Page fault. Frames: [1] [2] [3] [6]
Access page 2: Hit. Frames: [1] [2] [3] [6]
Access page 1: Hit. Frames: [1] [2] [3] [6]
Access page 2: Hit. Frames: [1] [2] [3] [6]
Access page 3: Hit. Frames: [1] [2] [3] [6]
Access page 7: Page fault. Frames: [7] [2] [3] [6]
Access page 6: Hit. Frames: [7] [2] [3] [6]
Access page 3: Hit. Frames: [7] [2] [3] [6]
Access page 2: Hit. Frames: [7] [2] [3] [6]
Access page 1: Page fault. Frames: [1] [2] [3] [6]
Access page 2: Hit. Frames: [1] [2] [3] [6]
Access page 3: Hit. Frames: [1] [2] [3] [6]
Access page 6: Hit. Frames: [1] [2] [3] [6]
The number of page faults of OPT is: 7
Page fault rate: 0.3889
The number of replacements: 3
Replacement rate: 0.1667
0
hamidur@DESKTOP-7CGRQ8S:~/LUT_exercises/os_sp/ex5$
```

Figure 3. Terminal showing run command with the resulting output of OPT page replacement

OPT gave the fewest page faults because it always replaces the page that will be needed farthest in the future. Since it uses future information, it provides the best possible result for this page reference sequence.

In this case, the result was 7 page faults, so the page fault rate was 0.3889. It also made 3 replacements and a replacement rate of 0.1667.

Differences in the page fault rates of the three replacement algorithms

Algorithm	Page faults	Page fault rate	Replacements	Replacement rate
FIFO	13	0.7222	9	0.5000
LRU	10	0.5556	6	0.3333
OPT	7	0.3889	3	0.1667

Table 1. Comparison of page fault rates and replacement rates

The page fault rate was highest in FIFO, lower in LRU, and lowest in OPT. This means the better an algorithm can judge which page will be needed later, the better it performs. FIFO does not look at usage pattern at all. LRU uses recent access history, while OPT uses exact future information. That is why the performance improves in the order FIFO, LRU, and OPT.

So, for this reference string, FIFO was the weakest, LRU performed better, and OPT gave the best result. FIFO is easy to implement, but it is less efficient. LRU is more practical because it uses recent usage information. OPT gives the ideal result and is mainly used as a benchmark for comparison.